UDA 6 · LA PROGRAMMAZIONE IMPERATIVA IN C E C++

Soluzioni: programmare in C e C++

Punti per ogni voce e indicazioni per i casi dubbi. Output verificati con gcc.

Dal punteggio al voto: voto in decimi = punti ÷ 10 (es. 73 punti → 7,3; 55 punti → 5,5), arrotondato secondo la griglia del dipartimento.

1. Scelta multipla

10 punti · 2 per voce

a) **Dennis Ritchie** · b) **Traduce il sorgente in linguaggio macchina** · c) `numero_2` · d) `scanf("%f", &x);` · e) **Da 1 a 10**

Più risposte segnate nella stessa domanda: 0 punti.

2. Vero o falso

10 punti · 2 per voce

F (`=` assegna, `==` confronta) · **V** (divisione intera) · **V** · **F** (se la condizione è subito falsa il while fa zero giri) · **V**

3. Completa il programma

8 punti · 2 per spazio

stdio.h · **&** (cioè `&a`) · **%d** · **return**

4. Che cosa stampa?

12 punti · 3 per frammento

a) `3 2` · b) `3.50` · c) `9 4` · d) `B`

Note: in a) 1,5 punti se è giusto un solo valore. In b) "3.5" vale 2 punti (risultato giusto, formato sbagliato). In d) `x = 0` è un'assegnazione: vale 0, quindi la condizione è falsa e si esegue l'else; chi risponde "A" o "errore" prende 0.

5. Le condizioni

10 punti · 2,5 per voce

a) `n >= 1 && n <= 100` · b) `x % 2 == 0 && x > 0` · c) `c == 'S' || c == 's'` · d) `voto < 1 || voto > 10`

Accettare forme equivalenti (per esempio `!(voto >= 1 && voto <= 10)`). `1 <= n <= 100` vale 0 punti: in C compila ma è sempre vera. Un solo `=` al posto di `==`, o le virgolette doppie al posto degli apici in c): metà punteggio.

6. Il ciclo for

10 punti

Giro	1	2	3	4
i	2	4	6	8
s	2	6	12	20

Tabella: 4 punti (0,5 per cella). Output: `2 6 12 20` e, a capo, `i = 10` (2 punti). Giri: **4** (2 punti). Perché: **dopo l'ultimo giro il passo porta i a 10, la condizione `i <= 8` diventa falsa e il ciclo esce** (2 punti).

7. Cicli annidati e passo negativo

8 punti

Output (6 punti, 2 per riga): `***`, `**`, `*` su tre righe. Totale asterischi: **6** (2 punti). Chi disegna il triangolo al contrario (1, 2, 3) prende 3 punti.

8. Il ciclo while

10 punti

a) **n non cambia mai dentro il ciclo, quindi `n > 0` resta sempre vera: ciclo infinito. Correzione: aggiungere `n--;` (o `n = n - 1;`) dentro le graffe.** Spiegazione 3 punti, correzione 2 punti.

b) **c = 3, n = 3** (100 → 33 → 11 → 3). 2,5 punti per valore.

9. Il programma

16 punti

```
int n, i, sufficienti = 0;
float voto, somma = 0;
printf("Quanti voti (1-20)? ");
scanf("%d", &n);
while (n < 1 || n > 20) {
    printf("Non valido, riprova: ");
    scanf("%d", &n);
}
for (i = 1; i <= n; i++) {
    printf("Voto %d: ", i);
    scanf("%f", &voto);
    somma = somma + voto;
    if (voto >= 6) {
        sufficienti++;
    }
}
printf("Media: %.2f\n", somma / n);
printf("Sufficienti: %d\n", sufficienti);
```

Criterio	Punti
Lettura di n con controllo ripetuto (while con condizione corretta)	4
Ciclo for che legge esattamente n voti, con il tipo float e `&`	4
Accumulatore inizializzato a 0 e media stampata con `%.2f`	4
Contatore dei sufficienti con if e condizione `>= 6`	3
Struttura generale (dichiarazioni, punto e virgola, graffe)	1

Va bene anche la versione C++ con `cin` e `cout` (per i due decimali basta stampare il valore della media: non penalizzare l'assenza di `setprecision`). Se la somma è `int` e i voti sono interi, togliere 2 punti dal terzo criterio. Controllo con l'esempio del testo: voti 7, 5.5, 8, 6.5 → media 6.75, sufficienti 3.

10. Domanda aperta

6 punti

Risposta attesa: nell'**iterazione definita** il numero di ripetizioni è noto prima di cominciare (es. fare 20 flessioni, stampare la tabellina fino a 10): si usa il **for**. Nell'**iterazione indefinita** si ripete finché una condizione resta vera, senza sapere quante volte (es. mescolare finché l'acqua bolle, chiedere un voto finché non è valido): si usa il **while**. Punti: definizione definita 2, definizione indefinita 2, esempi corretti e ciclo giusto per ciascuno 2.